DB Week 12 Workshop

INFO20003 | Sandy Luo

Workshop Overview

01

NoSQL Database

02

Lab: NoSQL Database

What is NoSQL?

NoSQL Database

- **Non-relational** (non-tabular) database
  - More *flexible*
  - Not dependent on any structures (e.g. rows, columns) to organise data

- Adaptable to performance, scalability and flexibility requirements of modern data storage and analysis → increasingly intensive needs from applications

- Examples of unstructured, but exponentially-growing data:
  - Chat data, large objects such as videos and images...

01

Graph database

02

Key-value store

03

Column-family store

04

Document store

NoSQL Database Types

1. Graph Database

- Use the concept of a graph to store / connect / query data
  - ***Nodes*** are *equivalent* to rows (instances of entities)
  - Nodes are connected by ***edges***, resembling relational relationships

- Both nodes and edges can have properties associated with them

- Example: *neo4j*
  - Used by Airbnb, Microsoft, IBM, eBay and Walmart

Graph Database

01

Graph database

02

Key-value store

03

Column-family store

04

Document store

NoSQL Database Types

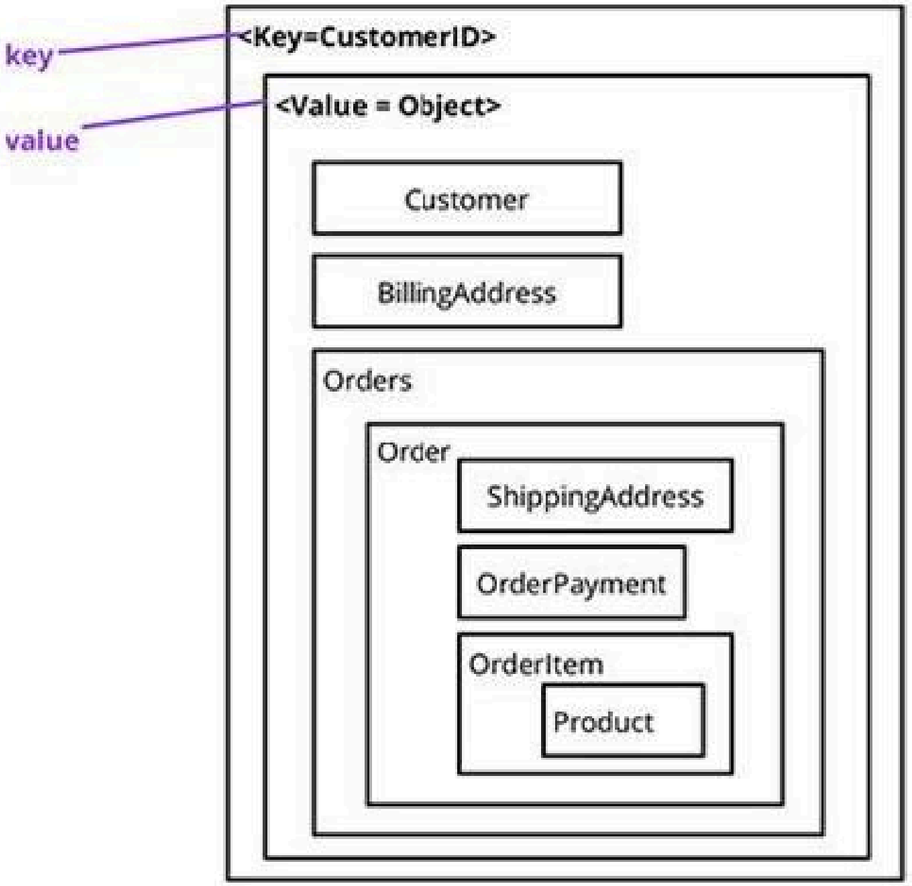
2. Key-value Store

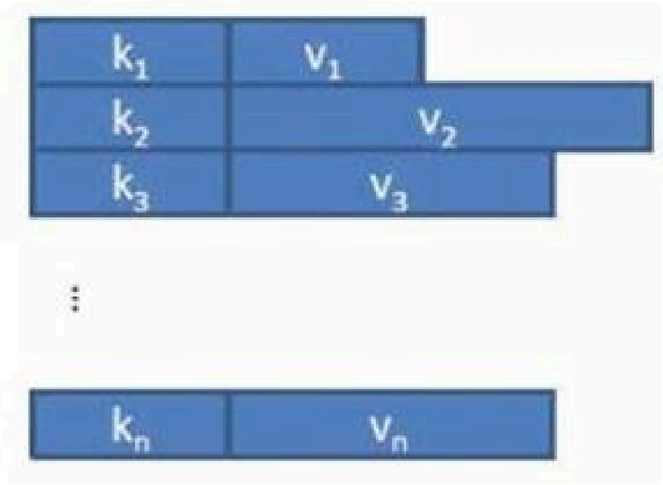
- Store data in key-value pairs
  - ***Key*** = Unique key (any data type, but some DBMS may have limitations)
  - ***Value*** = anything (e.g. number, array, image, JSON…)

- Application is in charge of interpreting what the key / value means
- No schema: Least structured -> most flexible, highly scalable

- Examples: Riak, Redis, Memcached, BerkeleyDB….

Key-value Store





01

Graph database

02

Key-value store

03

Column-family store

04

Document store

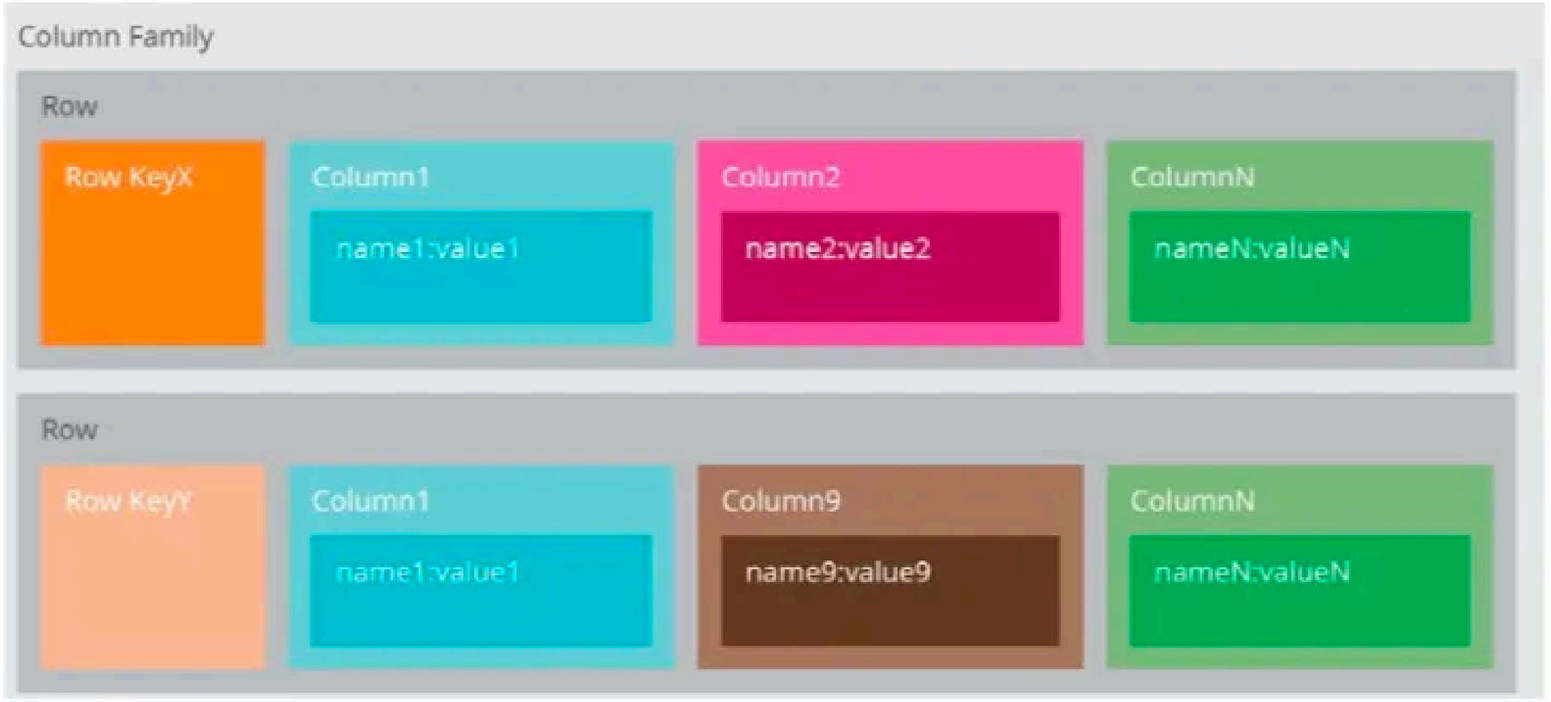
NoSQL Database Types

3. Column-family Store

- A type of **key-value** database: aka *wide-column* / *extensible record* stores
- Schema-free version of relational databases
  - Also uses tables, rows and columns
- **Columns are created for each row** instead of being predefined for a table
  - Columns rather than rows are stored together on disk
  - Makes analysis faster, as less data is fetched
- Related columns grouped together into “families”

- Examples: Cassandra, HBase

Column-family Store



01

Graph database

02

Key-value store

03

Column-family store

04

Document store

NoSQL Database Types

4. Document store

- Similar to **key-value** store
- Difference:
  - Value is a ***semi-structured*** or ***structured*** document
  - JSON, XML, BSON, industry-specific data formats

- No overall schema, each document can have its own structure and schema
  - Very flexible, indexible

- Example: MongoDB

Q1.

Libraries store information about their collections in their catalogue.

Match each of the following statements to the type of NoSQL database that would be best for storing that library's data. Select from the four types of NoSQL database discussed previously.

a. In one library, items are catalogued by author, title and publisher, as well as any number of other fields chosen by the cataloguer, such as physical description, subject codes and notes.
b. In another library, each catalogue record is stored in the MARC format (Figure 1), a coded text format that contains all the catalogue information for a particular item.
c. A public library wishes to store cover photos of all its items, which might be in JPEG, PNG or PDF format, or stored as a URL.
d. A university library wishes to keep track of which published academic papers reference each other in order to help researchers measure their metrics.

```
LEADER 00000nam  22000001  4500
008    730220s1955    ilu      b    00000 eng
019    55007351
050 0  QA276.5|b.R3
082    311.22
110 20 Rand Corporation.
245 12 A million random digits|bwith 100,000 normal deviates.
260 0  Glencoe, Ill.,|bFree Press|c[1955]
300    xxv, 400, 200 p.|c28 cm.
504    Bibliography: p. xxiv-xxv.
650  0 Numbers, Random.
984    |cMS T 519 R152
```

Figure 1: An example of a MARC record. MARC is a very old format that predates NoSQL, JSON and even XML by several decades, yet it remains the industry standard in library data systems.

Q1.(a)

a. In one library, items are catalogued by author, title and publisher, as well as any number of other fields chosen by the cataloguer, such as physical description, subject codes and notes.

- **Column-family** database
- Each row in a *column-family* table may have a different set of columns associated with it

Q1.(b)

b. In another library, each catalogue record is stored in the MARC format (Figure 1), a coded text format that contains all the catalogue information for a particular item.

```
LEADER 00000nam  22000001  4500
008    730220s1955    ilu      b    00000 eng
019    55007351
050 0  QA276.5|b.R3
082    311.22
110 20 Rand Corporation.
245 12 A million random digits|bwith 100,000 normal deviates.
260 0  Glencoe, Ill.,|bFree Press|c[1955]
300    xxv, 400, 200 p.|c28 cm.
504    Bibliography: p. xxiv-xxv.
650  0 Numbers, Random.
984    |cMS T 519 R152
```

Figure 1: An example of a MARC record. MARC is a very old format that predates NoSQL, JSON and even XML by several decades, yet it remains the industry standard in library data systems.

- **Document store**
- Generally use modern data interchange formats e.g. JSON / XML, but industry-specific data formats such as MARC can be used with specialised document store systems

Q1.(c)

c. A public library wishes to store cover photos of all its items, which might be in JPEG, PNG or PDF format, or stored as a URL.

- **Key-value** stores
  - Can store any kind of data

- NOT document store
  - Documents should be made up of structured data
  - Images are not structured data in the same way as JSON, unsuitable

Q1.(d)

d. A university library wishes to keep track of which published academic papers reference each other in order to help researchers measure their metrics.

- **Graph** database
- Store Relationships between papers
  - Papers as nodes, references as edges
  -

NoSQL Properties

01

Flexible Modelling

02

Scalability

03

Performance

04

High availability

NoSQL Advantages

1. Flexible Modelling

- **Flexible** data models
- Don’t rely on fixed schemas, data types, row size and column names

- Can cope better with less structured data sources, e.g. crowdsourced data

01

Flexible Modelling

02

Scalability

03

Performance

04

High availability

NoSQL Advantages

2. Scalability

- Capacity can be added/removed quickly → **horizontal scale-out methodology**
  - Adding inexpensive servers and connecting them to a database cluster

- Cost and complexity associated with scaling up a relational database into a distributed database are avoided
- More efficient when handling big data

01

Flexible Modelling

02

Scalability

03

Performance

04

High availability

NoSQL Advantages

3. Performance

- NoSQL databases are typically stored in partitions
  - ***Horizontal scale-out methodology***

- Users can be routed to closest data centre
- Efficient reads, writes and storage of the data items when handling big data

01

Flexible Modelling

02

Scalability

03

Performance

04

High availability

NoSQL Advantages

4. High Availability

- NoSQL databases are typically stored in partitions
  - Divide data across multiple DB instances without any shared resources
  - Not dependent on each other

- If nodes fail, DB can continue read and write operations on a different node

CAP Theorem

Theorem = A system can achieve only max. two out of three principles at once

1. **Consistency**
   - All the servers hosting the database will have the same data

2. **Availability**
   - The system will always respond to a request, whether consistent or not

3. **Partition Tolerance**
   - Partition = network partition
   - System continues to operate even if individual servers fail/cannot be reached

CAP Theorem

Theorem = A system can achieve only max. two out of three principles at once

- The biggest advantage for NoSQL DB is its ***partition tolerance*** → always keep P
  - Sacrifice consistency or availability

AP (- consistency)	**CP (- availability)**
• DB always answers, but possibly with outdated or wrong data • Achieve eventual consistency	DB stops all operations until the latest copy of data is available on all nodes
e.g. Allow reads before updating all the nodes	e.g. lock all the nodes before allowing reads

- Most choose AP > CP: ensure continuous availability and eventual consistency